МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №1

по курсу «Компьютерная графика»

Основы 3D графики

Вариант 8 — Цилиндр

Выполнил: Нгуен Шон
Группа: М8О-305БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы — построить простой трёхмерный объект и изучить его отображение на плоскость экрана с помощью матриц проекции и аффинных преобразований. По варианту 8 необходимо построить цилиндр. Программа должна работать в реальном времени и позволять изменять параметры объекта через интерфейс.

Для реализации использованы C++, Vulkan, GLFW и ImGui. В программе выполнены все шесть дополнительных заданий, предусмотренных для оценки «5»:

1. Переключение между перспективной и ортографической проекциями.

2. Изменение положения, поворота и масштаба фигуры.

3. Движение по сложной траектории и вращение с возможностью паузы, изменения скорости и радиуса.

4. Изменение цвета через интерфейс.

5. Процедурное задание цветов вершин по их локальным координатам и умножение на выбранный цвет.

6. Отрисовка двух объектов с использованием отдельных наборов дескрипторов VkDescriptorSet.

Метод решения

Построение цилиндра

Фигура задана в локальной системе координат, то есть до перемещения и поворота. Её центр находится в начале координат, ось симметрии направлена вдоль оси Y. Цилиндр задаётся радиусом R = 1,0 и высотой H = 2,0, поэтому занимает диапазон x, z ∈ [−1; 1] и y ∈ [−1; 1].

Основание цилиндра разбито на 50 сегментов. Для каждого сегмента вычисляется угол $\alpha = 2\pi \cdot i / 50$, после чего координаты точки на окружности находятся как $x = R \cdot \cos \alpha$, $z = R \cdot \sin \alpha$. Боковая поверхность состоит из 50 прямоугольников, каждый из которых разбит на два треугольника. Верхнее и нижнее основания — это «веера» из 50 треугольников, сходящихся в центре крышки. Итого: 300 вершин боковой поверхности + 300 вершин оснований. Каждая вершина хранит позицию и цвет.

```
struct Vertex { float position[3]; float color[3]; };
```

Однородные координаты

Обычная точка в пространстве задаётся тремя координатами (x, y, z). Для преобразований в программе к ним добавляется четвёртая компонента w. Исходная точка записывается как столбец (x, y, z, 1). Такая запись называется однородными координатами. Дополнительная компонента позволяет записать перенос как умножение на матрицу: матрица 3×3 подходит для поворота и масштабирования относительно начала координат, но не может добавить к каждой точке постоянное смещение. Матрица 4×4 объединяет все операции в одной форме.

Перенос

Матрица переноса T имеет вид: T = [[1,0,0,tx],[0,1,0,ty],[0,0,1,tz],[0,0,0,1]]. Умножение T на вектор (x, y, z, 1) даёт (x + tx, y + ty, z + tz, 1). Например, перенос на (2, 0, 0) переводит точку (1, 0, 0) в (3, 0, 0). Форма и размеры фигуры при этом не меняются.

Масштабирование

Матрица масштабирования S = [[sx,0,0,0],[0,sy,0,0],[0,0,sz,0],[0,0,0,1]]. После умножения координаты становятся (sx·x, sy·y, sz·z). Если все коэффициенты одинаковы, фигура равномерно увеличивается или уменьшается. Если изменить только sx, она растянется вдоль локальной оси X. В интерфейсе эти коэффициенты задаются полем Scale.

Поворот

Поворот задаётся углом и осью. Матрицы для поворота вокруг осей X, Y и Z имеют стандартный вид (c = cos α, s = sin α). При повороте вокруг X координата x не меняется, а y и z пересчитываются через синус и косинус. Аналогично работают повороты вокруг Y и Z. В интерфейсе углы указаны в градусах, перед вычислением используется перевод α = degrees·π/180.

Порядок преобразований

Матрица модели вычисляется как M = T_position · T_orbit · R_x · R_y · R_z · S. При умножении на вектор-столбец операции выполняются справа налево: сначала масштабирование, затем повороты вокруг X, Y и Z, после этого перенос по траектории и смещение, выбранное пользователем. Порядок важен: если сначала перенести фигуру, а затем повернуть относительно начала координат, изменится и положение её центра.

Камера и проекции

Камера фиксирована в точке (0, 0, 6) и направлена вдоль отрицательного направления оси Z. Матрица вида V = T(0, 0, −6) переводит мировые координаты в систему координат камеры. Итоговая матрица MVP = P · V · M применяется к каждой вершине в вершинном шейдере:

```
gl_Position = scene.mvp * vec4(inPosition, 1.0);
```

Результат — координаты отсечения. Затем графический конвейер выполняет деление x, y и z на w и преобразует координаты в положение на экране. Самостоятельно это деление в шейдере не выполняется.

Перспективная проекция создаёт эффект глубины: более удалённые объекты выглядят меньше. В коде вертикальный угол обзора равен 45°, ближняя плоскость n = 0,1, дальняя — F = 100. Ортографическая проекция не зависит от расстояния до камеры: высота видимой области H = 4, ширина W = H·a, где a — отношение сторон окна. Обе матрицы рассчитаны на диапазон глубины Vulkan от 0 до 1 (используется GLM_FORCE_DEPTH_ZERO_TO_ONE). Переключатель Perspective projection выбирает нужную матрицу.

Анимация по траектории

Положение центра фигуры меняется с фазой φ. Каждый кадр фаза увеличивается на speed · Δt, где Δt — время между обновлениями. При паузе фаза не изменяется. Координаты центра вычисляются как: x = r·cos φ, z = r·sin φ, где r — радиус орбиты, настраиваемый через поле Orbit radius. Фигура дополнительно вращается вокруг оси Y: угол поворота пропорционален фазе. Поля Orbit speed и Orbit radius позволяют менять скорость и радиус траектории. При паузе можно вручную менять Position, Rotation и Scale.

Процедурные цвета вершин

Каждая вершина получает RGB-цвет по своим локальным координатам: компонент R зависит от y (высоты), компонент G — обратно пропорционален y, компонент B — константа. Формула переводит диапазон y ∈ [−1; 1] в значения от 0 до 1:

```
t = (y + 1) / 2;  color = vec3(t, 1 – t, 0.5);
```

Между вершинами цвет интерполируется по поверхности треугольника. Выбранный через Color multiplier цвет покомпонентно умножается на цвет вершины во фрагментном шейдере: outColor = userColor * vec4(fragColor, 1.0). Например, множитель (1, 0, 0) оставляет только красную составляющую. Освещение в этой реализации не рассчитывается.

Объекты Vulkan и отрисовка

Координаты и цвета вершин записываются в вершинный буфер VkBuffer, а номера вершин треугольников — в индексный буфер. Память буферов выделяется с помощью VMA. Uniform-буфер содержит матрицу MVP (model, view, proj) и цветовой множитель tint, которые обновляются перед отрисовкой.

Для двух объектов создаются два uniform-буфера и два набора VkDescriptorSet. Набор дескрипторов связывает шейдер с uniform-буфером конкретного объекта. VkDescriptorSetLayout описывает эту связь, VkDescriptorPool используется для выделения наборов, а VkPipelineLayout задаёт их использование в конвейере.

Графический конвейер VkPipeline настроен на список треугольников. Вершинный шейдер преобразует координаты и вычисляет цвет вершины. Фрагментный шейдер выводит интерполированный цвет. Проверка глубины позволяет ближним поверхностям перекрывать дальние независимо от порядка отрисовки.

Перед каждым вызовом рисования привязывается свой набор дескрипторов. Поэтому один и тот же вершинный и индексный буфер можно использовать повторно, но получать объекты в разных положениях.

Результаты

Ниже приведены снимки работающей программы. Для сравнения проекций удобно включить Pause animation и сохранить одинаковые положение, поворот и масштаб. Перспективный и ортографический виды могут отличаться общим размером фигуры: в коде угол обзора и размер ортографической области заданы отдельно.

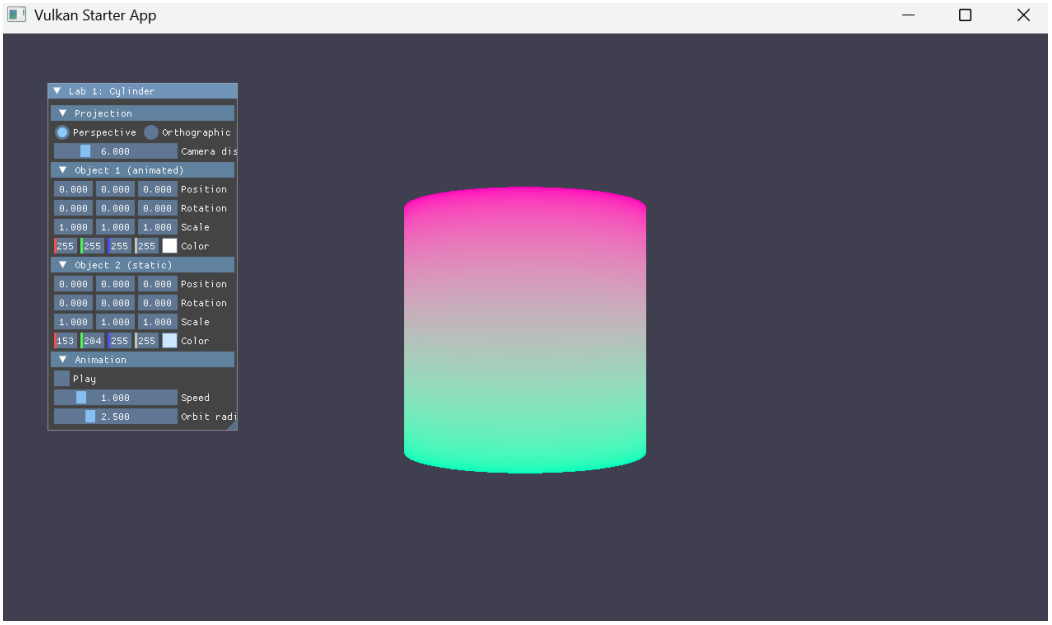


Рисунок 1 — Перспективная проекция

Один цилиндр; Perspective projection включён. На снимке показан вид фигуры с перспективным уменьшением по глубине. Виден процедурный градиент цвета по высоте цилиндра.

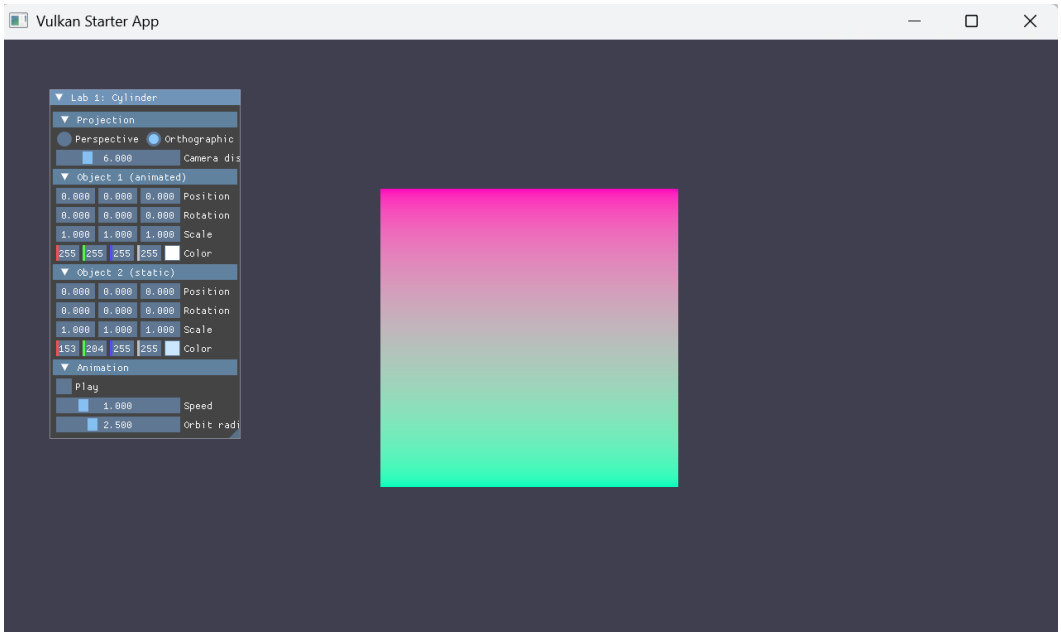


Рисунок 2 — Ортографическая проекция

Perspective projection выключен. Положение и поворот сохранены такими же, как на предыдущем снимке; размер по проекции не зависит от глубины.

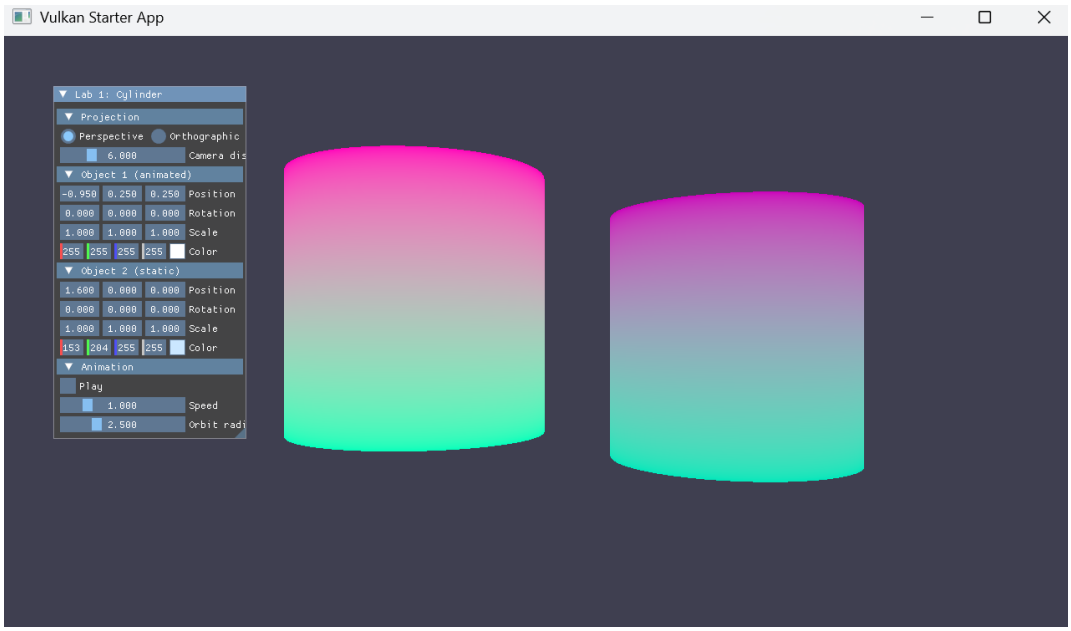


Рисунок 3 — Два объекта в сцене

Два цилиндра используют общую геометрию и отдельные наборы дескрипторов с разными матрицами MVP.

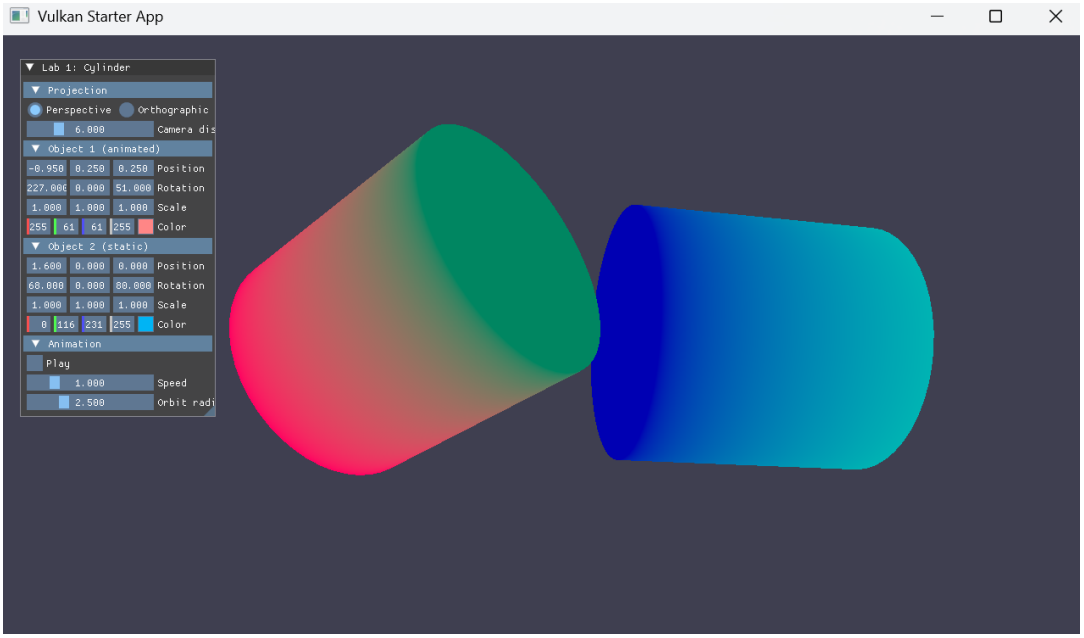


Рисунок 4 — Изменение ориентации и цвета

Через Rotation изменены углы поворота, через Color multiplier — цветовой множитель. Показан результат изменения параметров через интерфейс ImGui.

Выводы

В работе я построил цилиндр из треугольников и реализовал его отображение с помощью Vulkan. Для изменения положения, ориентации и размеров использовал матрицы 4×4. Разобрал порядок их применения и различие между перспективной и ортографической проекциями.

Добавил управление параметрами через ImGui, движение по круговой траектории с настройкой радиуса и скорости, паузу и изменение цвета. Для отображения двух объектов использовал отдельные uniform-буферы и наборы дескрипторов. Все шесть дополнительных заданий реализованы в программе.

Я научился работать с графическим конвейером Vulkan, настраивать матрицы преобразований и проекций, использовать дескрипторы для передачи данных в шейдеры и организовывать интерактивное управление через ImGui.